

Material de Patrones de Diseño GoF para la Técnica de Aronson

Prof. Daniel San Martín

2 de abril de 2025

1. Patrones Creacionales

1.1. Abstract Factory

1.1.1. Descripción

El objetivo del patrón Abstract Factory es la creación de objetos agrupados en familias sin tener que conocer las clases concretas destinadas a la creación de estos objetos.

1.1.2. Ejemplo

El sistema de venta de vehículos gestiona vehículos que funcionan con gasolina y vehículos eléctricos. Esta gestión está delegada en el objeto Catálogo encargado de crear tales objetos.

Para cada producto, disponemos de una clase abstracta, de una subclase concreta derivando una versión del producto que funciona con gasolina y de una subclase concreta derivando una versión del producto que funciona con electricidad. Por ejemplo, en la Figura 1, para el objeto Scooter, existe una clase abstracta **Scooter** y dos subclases concretas **ScooterElectricidad** y **ScooterGasolina**.

El objeto **Catálogo** puede utilizar estas subclases concretas para instanciar los productos. No obstante si fuera necesario incluir nuevas clases de familias de vehículos (diésel o mixto gasolina-eléctrico), las modificaciones a realizar en el objeto **Catálogo** pueden ser bastante pesadas.

El patrón Abstract Factory resuelve este problema introduciendo una interfaz **FabricaVehículo** que contiene la firma de los métodos para definir cada producto. El tipo devuelto por estos métodos está constituido por una de las clases abstractas del producto. De este modo el objeto **Catálogo** no necesita conocer las subclases concretas y permanece desacoplado de las familias de producto.

Se incluye una subclase de implementación de **FabricaVehículo** por cada familia de producto, a saber las subclases **FabricaVehículoElectricidad** y **FabricaVehículoGasolina**. Dicha subclase implementa las operaciones de creación del vehículo apropiado para la familia a la que está asociada.

El objeto **Catálogo** recibe como parámetro una instancia que responde a la interfaz **FabricaVehículo**, es decir o bien una instancia de **FabricaVehículoElectricidad**, o bien una instancia de **FabricaVehículoGasolina**. Con dicha instancia, el catálogo puede crear y manipular los vehículos sin tener que conocer las familias de vehículos y las clases concretas de instanciación correspondientes.

El conjunto de clases del patrón Abstract Factory para este ejemplo se detalla en la Figura 1.

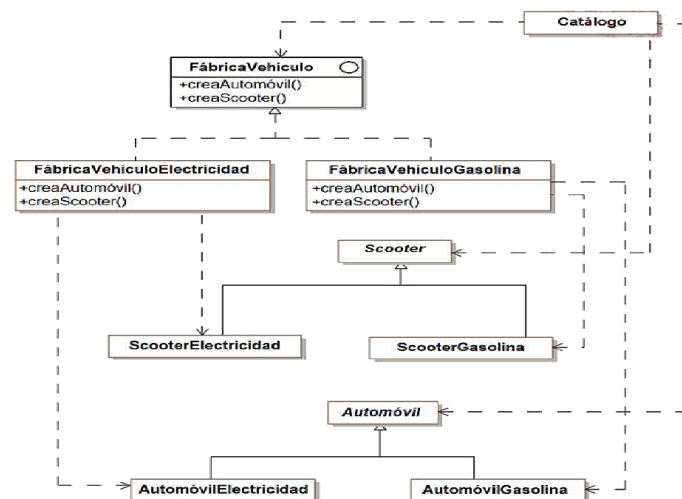


Figura 1: El patrón Abstract Factory aplicado a las familias de vehículos

1.1.3. Estructura

La figura 2 detalla la estructura genérica del patrón.

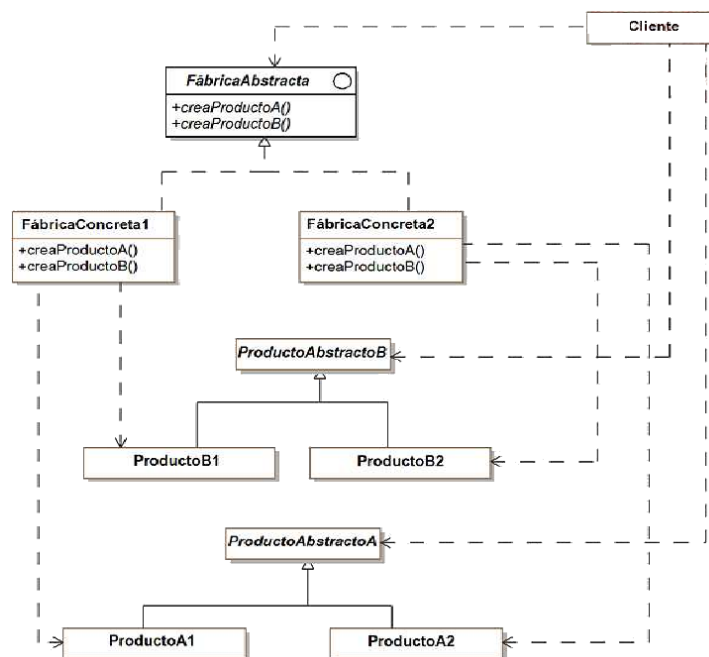


Figura 2: Estructura del patrón Abstract Factory

1.1.4. Participantes

Los participantes del patrón son los siguientes:

- **FábricaAbstracta** (**FábricaVehículo**) es una interfaz que define las firmas de los métodos que crean los distintos productos.
- **FábricaConcreta1**, **FábricaConcreta2** (**FábricaVehículoElectricidad**, **FábricaVehículoGasolina**) son las clases concretas que implementan los métodos que crean los productos para cada familia de producto. Conociendo la familia y el producto, son capaces de crear una instancia del producto para esta familia.
- **ProductoAbstractoA** y **ProductoAbstractoB** (**Scooter** y **Automóvil**) son las clases abstractas de los productos independientemente de su familia. Las familias se introducen en las subclases concretas.
- **Cliente** es la clase que utiliza la interfaz de **FábricaAbstracta**.

1.1.5. Colaboraciones

La clase **Cliente** utiliza una instancia de una de las fábricas concretas para crear sus productos a partir de la interfaz **FabricaAbstracta**.

Normalmente sólo es necesario crear una instancia de cada fábrica concreta, que puede compartirse por varios clientes.

1.1.6. Dominio de Uso

El patrón se utiliza en los dominios siguientes:

- Un sistema que utiliza productos necesita ser independiente de la forma en que se crean y agrupan estos productos.
- Un sistema está configurado según varias familias de productos que pueden evolucionar.

1.1.7. Ejemplo Java

Se presenta a continuación un pequeño ejemplo de uso del patrón escrito en Java. El código Java correspondiente a la clase abstracta **Automovil** y sus subclases aparece a continuación. Es muy sencillo, describe los cuatro atributos de los automóviles así como el método **mostrarCaracteristicas** que permite visualizarlas.

```

1 public abstract class Automovil {
2     protected String modelo;
3     protected String color;
4
5     protected int potencia;
6     protected double espacio;
7
8     public Automovil(String modelo, String color, int potencia, double espacio) {
9         this.modelo = modelo;
10        this.color = color;
11        this.potencia = potencia;
12        this.espacio = espacio;
13    }
14    public abstract void mostrarCaracteristicas();
15 }
16
17 public class AutomovilElectricidad extends Automovil {
18     public AutomovilElectricidad(String modelo, String
19     color, int potencia, double espacio) {
20         super(modelo, color, potencia, espacio);
21     }
22     public void mostrarCaracteristicas() {
23         System.out.println(
24             "Automovil electrico de modelo: " + modelo + " de color: " + color + " de potencia: " +
25             potencia + " de espacio: " + espacio);
26     }
27 }
28
29 public class AutomovilGasolina extends Automovil {
30     public AutomovilGasolina(String modelo, String
31     color, int potencia, double espacio) {
32         super(modelo, color, potencia, espacio);
33     }
34     public void mostrarCaracteristicas() {
35         System.out.println(
36             "Automovil de gasolina de modelo: " + modelo + " de color: " + color + " de potencia: " +
37             potencia + " de espacio: " + espacio);
38     }
39 }

```

El código Java correspondiente a la clase abstracta Scooter y sus subclases aparece a continuación. Es similar al de los automóviles, salvo por el atributo espacio que no existe para las scooters.

```

1 public abstract class Scooter {
2     protected String modelo;
3     protected String color;
4     protected int potencia;
5
6     public Scooter(String modelo, String color, int potencia) {
7         this.modelo = modelo;
8         this.color = color;
9         this.potencia = potencia;
10    }
11    public abstract void mostrarCaracteristicas();
12 }
13
14 public class ScooterElectricidad extends Scooter {
15     public ScooterElectricidad(String modelo, String color, int potencia) {
16         super(modelo, color, potencia);
17     }
18     public void mostrarCaracteristicas() {
19         System.out.println("Scooter electrica de modelo: " + modelo + " de color: " + color +
20             " de potencia: " + potencia);
21     }
22 }
23
24 public class ScooterGasolina extends Scooter {
25     public ScooterGasolina(String modelo, String color, int potencia) {
26         super(modelo, color, potencia);
27     }
28     public void mostrarCaracteristicas() {
29         System.out.println("Scooter de gasolina de modelo: " + modelo + " de color: " + color +
30             " de potencia: " + potencia);
31     }
32 }

```

Ahora es posible introducir la interfaz `FabricaVehiculo` y sus dos clases de implementación, una para cada familia (eléctrico/gasolina). Es fácil darse cuenta que solo las clases de implementación utilizan las clases concretas de los vehículos.

```
1 public interface FabricaVehiculo {
2     Automovil creaAutomovil(String modelo, String color, int potencia, double espacio);
3     Scooter creaScooter(String modelo, String color, int potencia);
4 }
5
6 public class FabricaVehiculoElectricidad implements FabricaVehiculo {
7     public Automovil creaAutomovil(String modelo, String color, int potencia, double espacio) {
8         return new AutomovilElectricidad(modelo, color, potencia, espacio);
9     }
10    public Scooter creaScooter(String modelo, String color, int potencia) {
11        return new ScooterElectricidad(modelo, color, potencia);
12    }
13 }
14 public class FabricaVehiculoGasolina implements FabricaVehiculo {
15     public Automovil creaAutomovil(String modelo, String color, int potencia, double espacio) {
16         return new AutomovilGasolina(modelo, color, potencia, espacio);
17     }
18     public Scooter creaScooter(String modelo, String color, int potencia) {
19         return new ScooterGasolina(modelo, color, potencia);
20     }
21 }
```

Por último, se presenta el código fuente Java del cliente de la fábrica, a saber el catálogo que es, en nuestro ejemplo, el programa principal. Por motivos de simplicidad, el catálogo solicita al comienzo la fábrica que se quiere utilizar (electricidad o gasolina). Esta fábrica debería proporcionarse como parámetro al catálogo.

El resto del programa es totalmente independiente de la familia de objetos, respetando el objetivo del patrón Abstract Factory.

```
1 import java.util.*;
2 public class Catalogo {
3
4     public static int nAutos = 3;
5     public static int nScooters = 2;
6
7     public static void main(String[] args) {
8         Scanner reader = new Scanner(System.in);
9         FabricaVehiculo fabrica;
10        Automovil[] autos = new Automovil[nAutos];
11        Scooter[] scooters = new Scooter[nScooters];
12        System.out.print("Desea utilizar " +
13            "vehiculos electricos (1) o a gasolina (2):");
14        String eleccion = reader.next();
15        if (eleccion.equals("1")) {
16            fabrica = new FabricaVehiculoElectricidad();
17        }
18        else {
19            fabrica = new FabricaVehiculoGasolina();
20        }
21        for (int index = 0; index < nAutos; index++)
22            autos[index] = fabrica.creaAutomovil("estandar", "amarillo", 6+index, 3.2);
23        for (int index = 0; index < nScooters; index++)
24            scooters[index] = fabrica.creaScooter("clasico", "rojo", 2+index);
25        for (Automovil auto: autos)
26            auto.mostrarCaracteristicas();
27        for (Scooter scooter: scooters)
28            scooter.mostrarCaracteristicas();
29    }
30 }
```

A continuación se muestra un ejemplo de ejecución para vehículos eléctricos:

```
Desea utilizar vehiculos electricos (1)
o a gasolina (2): 1
Automovil electrico de modelo: estandar de color:
amarillo de potencia: 6
de espacio: 3.2
Automovil electrico de modelo: estandar de color:
amarillo de potencia: 7
de espacio: 3.2
Automovil electrico de modelo: estandar de color:
```

amarillo de potencia: 8
de espacio: 3.2
Scooter electrica de modelo: clasico de color:
rojo de potencia: 2
Scooter electrica de modelo: clasico de color:
rojo de potencia: 3

En este ejemplo de ejecución se ha creado una fábrica de vehículos eléctricos, de modo que el catálogo se compone de automóviles y scooters eléctricos.

1.2. Factory Method

1.2.1. Descripción

El objetivo del patrón Factory Method es proveer un método abstracto de creación de un objeto delegando en las subclases concretas su creación efectiva.

1.2.2. Ejemplo

Vamos a centrarnos en los clientes y sus pedidos. La clase **Cliente** implementa el método **creaPedido** que debe crear el pedido. Ciertos clientes solicitan un vehículo pagando al contado y otros clientes utilizan un crédito. En función de la naturaleza del cliente, el método **creaPedido** debe crear una instancia de la clase **PedidoContado** o una instancia de la clase **PedidoCrédito**. Para realizar estas alternativas, el método **creaPedido** es abstracto. Ambos tipos de cliente se distinguen mediante dos subclases concretas de la clase abstracta **Cliente**:

- La clase concreta **ClienteContado** cuyo método **creaPedido** crea una instancia de la clase **PedidoContado**.
- La clase concreta **ClienteCredito** cuyo método **creaPedido** crea una instancia de la clase **PedidoCredito**.

Tal diseño está basado en el patrón Factory Method, el método **creaPedido** es el método de fabricación. El ejemplo se detalla en la Figura 3

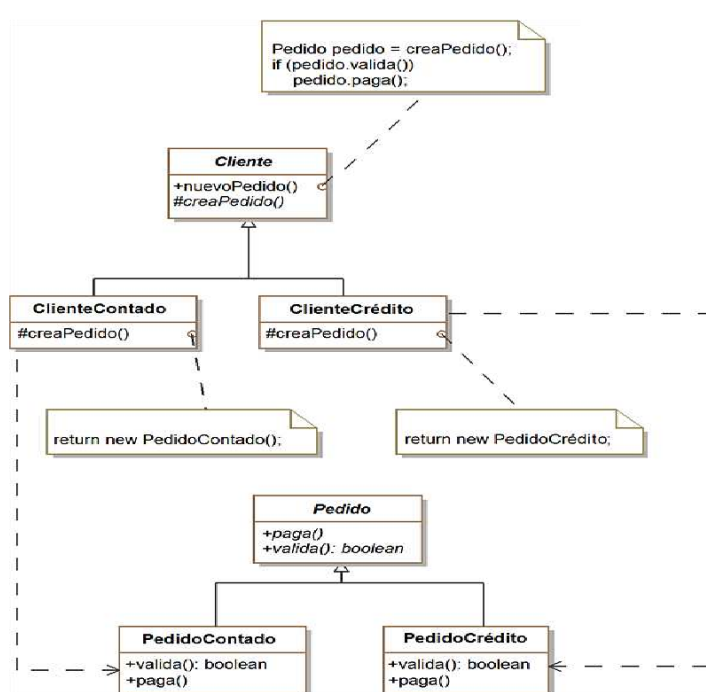


Figura 3: El patrón Factory Method aplicado a los clientes y sus pedidos

1.2.3. Estructura

La Figura 6.2 detalla la estructura genérica del patrón.

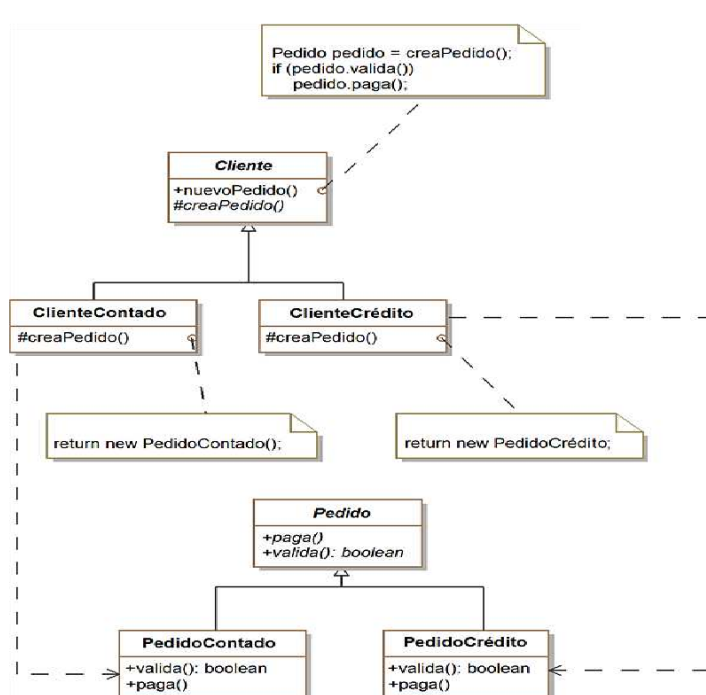


Figura 4: Estructura del patrón Factory Method

1.2.4. Participantes

Los participantes del patrón son los siguientes:

- **CreadorAbstracto** (**Cliente**) es una clase abstracta que implementa la firma del método de fabricación y los métodos que invocan al método de fabricación.
- **CreadorConcreto** (**ClienteContado**, **ClienteCredito**) es una clase concreta que implementa el método de fabricación. Pueden existir varios creadores concretos.
- **Producto** (**Pedido**) es una clase abstracta que describe las propiedades comunes de los productos.
- **ProductoConcreto** (**PedidoContado**, **PedidoCrédito**) es una clase concreta que describe completamente un producto.

1.2.5. Colaboraciones

Los métodos concretos de la clase **CreadorAbstracto** se basan en la implementación del método de fabricación en las subclases. Esta implementación crea una instancia de la subclase adecuada de **Producto**.

1.2.6. Dominios de Uso

El patrón se utiliza en los casos siguientes:

- Una clase que sólo conoce los objetos con los que tiene relaciones.
- Una clase quiere transmitir a sus subclases las elecciones de instanciación aprovechando un mecanismo de polimorfismo.

1.2.7. Ejemplo de Java

El código fuente de la clase abstracta **Pedido** y de sus dos subclases concretas aparece a continuación. El importe del pedido se pasa como parámetro al constructor de la clase. Si la validación de un pedido al contado es sistemática, tenemos la posibilidad de escoger, para nuestro ejemplo, aceptar únicamente aquellos pedidos provistos de un crédito cuyo valor se sitúe entre 1,000 y 5,000.

```

1 public abstract class Pedido {
2
3     protected double importe;
4
5     public Pedido(double importe) {
6         this.importe = importe;
7     }
8
9     public abstract boolean valida();
10    public abstract void paga();
11 }
12
13 public class PedidoContado extends Pedido
14 {
15     public PedidoContado(double importe) {
16         super(importe);
17     }
18     public void paga() {
19         System.out.println("El pago del pedido por importe de: " + importe + " se ha realizado.");
20     }
21
22     public boolean valida() {
23         return true;
24     }
25 }
26
27 public class PedidoCredito extends Pedido {
28
29     public PedidoCredito(double importe) {
30         super(importe)
31     }
32
33     public void paga() {
34         System.out.println("El pago del pedido a credito de: " + importe + " se ha realizado.");
35     }
36
37     public boolean valida() {
38         return (importe >= 1000.0) && (importe <= 5000.0);
39     }
40 }

```

El código fuente de la clase abstracta `Cliente` y de sus subclases concretas aparece a continuación. Un cliente puede realizar varios pedidos, y sólo los que se validan se agregan en su lista.

```

1 import java.util.*;
2 public abstract class Cliente {
3     protected List<Pedido> pedidos = new ArrayList<Pedido>();
4     protected abstract Pedido creaPedido(double importe);
5
6     public void nuevoPedido(double importe) {
7         Pedido pedido = this.creaPedido(importe);
8         if (pedido.valida()) {
9             pedido.paga();
10            pedidos.add(pedido);
11        }
12    }
13 }
14
15 public class ClienteContado extends Cliente {
16     protected Pedido creaPedido(double importe) {
17         return new PedidoContado(importe);
18     }
19 }
20
21 public class ClienteCredito extends Cliente {
22     protected Pedido creaPedido(double importe) {
23         return new PedidoCredito(importe);
24     }
25 }

```

Por último, la clase `Usuario` muestra un ejemplo de uso del patrón Factory Method. El nombre `Usuario` denota aquí un objeto usuario de un patrón.

```

1 public class Usuario {
2     public static void main(String[] args) {
3         Cliente cliente;

```

```
4         cliente = new ClienteContado();
5         cliente.nuevoPedido(2000.0);
6         cliente.nuevoPedido(10000.0);
7         cliente = new ClienteCredito();
8         cliente.nuevoPedido(2000.0);
9         cliente.nuevoPedido(10000.0);
10    }
11 }
```

Un ejemplo de ejecución del usuario daría la salida siguiente:

El pago del pedido por importe de: 2000 se ha realizado.
El pago del pedido por importe de: 10000 se ha realizado.
El pago del pedido a credito de: 2000 se ha realizado.

Se puede constatar que la solicitud de un pedido provisto de un crédito por valor de 10,000 ha sido rechazada.

1.3. Singleton

1.3.1. Descripción

El patrón Singleton tiene como objetivo asegurar que una clase sólo posee una instancia y proporcionar un método de clase único que devuelva esta instancia. En ciertos casos es útil gestionar clases que posean una única instancia. En el marco de los patrones de construcción, podemos citar el caso de una fábrica de productos (patrón Abstract Factory) del que sólo es necesario crear una instancia.

1.3.2. Ejemplo

En el sistema de venta online de vehículos, debemos gestionar clases que poseen una sola instancia. El sistema de documentación que debe entregarse al cliente tras la compra de un vehículo (como el certificado de cesión, la solicitud de matriculación y la orden de pedido) utiliza la clase `DocumentaciónEnBlanco` que sólo posee una instancia. Esta instancia referencia todos los documentos necesarios para el cliente. Esta instancia única se llama la documentación en blanco, pues los documentos a los que hace referencia están todos en blanco.

La figura 5 ilustra el uso del patrón Singleton para la clase `DocumentaciónEnBlanco`. El atributo de clase `instance` contiene o bien `null` o bien la única instancia de la clase `DocumentaciónEnBlanco`. El método de clase `Instance` reenvía esta instancia única devolviendo el valor del atributo `instance`. Si este atributo vale `null`, se inicializa previamente mediante la creación de la instancia única.

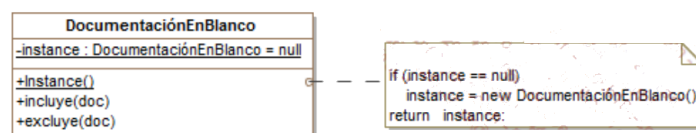


Figura 5: Patrón Singleton aplicado a la clase `DocumentaciónEnBlanco`

1.3.3. Estructura

La Figura 6 detalla la estructura genérica del patrón.

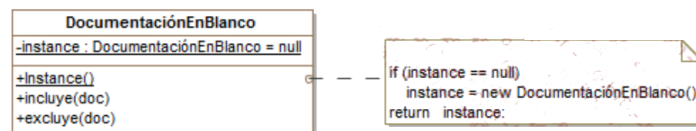


Figura 6: Estructura del Patrón Singleton

- **Participante:** El único participante es la clase Singleton, que ofrece acceso a la instancia única mediante el método de clase `Instance`. Por otro lado, la clase Singleton posee un mecanismo que asegura que sólo puede existir una única instancia. Este mecanismo bloquea la creación de otras instancias.
- **Colaboración:** Cada cliente de la clase Singleton accede a la instancia única mediante el método de clase `Instance`. No puede crear nuevas instancias utilizando el operador habitual de instanciación (operador `new`), que está bloqueado.

1.3.4. Dominio de Uso

El patrón se utiliza en el siguiente caso:

- Sólo debe existir una única instancia de una clase.
- Esta instancia sólo debe estar accesible mediante un método de clase.

1.3.5. Ejemplos Java

Clase `DocumentacionEnBlanco`.

El constructor de esta clase tiene una visibilidad privada de modo que sólo pueda utilizarlo el método `Instance`. De este modo, ningún objeto externo a la clase `DocumentaciónEnBlanco` puede crear una instancia utilizando el operador `new`. Del mismo modo, el atributo `_instance` también tiene una visibilidad privada para que sólo sea posible acceder a él desde el método de clase `Instance`.

```

1 import java.util.*;
2 public class DocumentacionEnBlanco extends Documentacion {
3     private static DocumentacionEnBlanco _instance = null;
4     private DocumentacionEnBlanco() {
5         documentos = new ArrayList<Documento>();
6     }
7 }
8 public static DocumentacionEnBlanco Instance() {
9     if (_instance == null)
10         _instance = new DocumentacionEnBlanco();
11 }
12 return _instance;
13 ...

```

El único cliente de la clase `DocumentacionEnBlanco` es la clase `DocumentacionCliente` que, en su constructor, obtiene una referencia a la documentación en blanco invocando al método `Instance`. A continuación, el constructor accede a la lista de documentos en blanco.

```

1 DocumentacionEnBlanco documentacionEnBlanco = DocumentacionEnBlanco.Instance();
2 List<Documento> documentosEnBlanco = documentacionEnBlanco.getDocumentos();

```

Clase Comercial.

En el sistema de venta de vehículos, queremos representar el vendedor mediante una clase que permita memorizar su información en lugar de utilizar variables globales que contienen respectivamente su nombre, su dirección, etc.

```

1 public class Comercial {
2     protected String nombre;
3     protected String direccion;
4     protected String email;
5     private static Comercial _instance = null;
6     private Comercial(){}
7     public static Comercial Instance() {
8         if (_instance == null)
9             _instance = new Comercial();
10        return _instance;
11    }
12    public void visualiza() {
13        System.out.println("Nombre: " + nombre);
14        System.out.println("Direccion: " + direccion);
15        System.out.println("Email: " + email);
16    }
17    //getters and setters
18    ..
19    ..
20 }

```

El programa principal siguiente utiliza la clase `Comercial`.

```

1 public class TestComercial {
2     public static void main(String[] args) {
3         // inicializacion del comercial en el sistema
4         Comercial elComercial = Comercial.Instance();
5         elComercial.setNombre("Comercial Coquimbo");
6         elComercial.setDireccion("Coquimbo");
7         elComercial.setEmail("comercial@comercialcoquimbo.cl");
8         // muestra el comercial del sistema
9         visualiza();
10    }
11    public static void visualiza() {
12        Comercial elComercial = Comercial.Instance();
13        elComercial.visualiza();
14    }
15 }

```

Su ejecución muestra que sólo existe una instancia debido a que el método `visualiza` de `TestComercial` no recibe ningún parámetro.

```

Nombre: Comercial Coquimbo
Dirección: Coquimbo
Email: comercial@comercialcoquimbo.cl

```

1.4. Builder

1.4.1. Descripción

El objetivo del patrón Builder es abstraer la construcción de objetos complejos de su implementación, de modo que un cliente pueda crear objetos complejos sin tener que preocuparse de las diferencias en su implantación.

1.4.2. Ejemplo

Durante la compra de un vehículo, el vendedor crea todo un conjunto de documentos que contienen en especial la solicitud de pedido y la solicitud de matriculación del cliente. Es posible construir estos documentos en formato HTML o en formato PDF según la elección del cliente. En el primer caso, el cliente le provee una instancia de la clase `ConstructorDocumentaciónVehiculo`, y en el segundo caso, una instancia de la clase `ConstructorDocumentaciónVehiculoPdf`. El vendedor realiza, a continuación, la solicitud de creación de cada documento mediante esta instancia.

De este modo el vendedor genera la documentación con ayuda de estos dos métodos: `construyeSolicitudPedido` y `construyeSolicitudMatriculacion`.

El conjunto de clases del patrón Builder para este ejemplo se detalla en la Figura 7. Esta figura muestra la jerarquía entre las clases `ConstructorDocumentaciónVehiculo` y Documentación. El vendedor puede crear las solicitudes de pedido y las solicitudes de matriculación sin conocer las subclases de `ConstructorDocumentaciónVehiculo` ni las de Documentación.

Las relaciones de dependencia entre el cliente y las subclases de `ConstructorDocumentaciónVehiculo` se explican por el hecho de que el cliente crea una instancia de estas subclases.

La estructura interna de las subclases concretas de Documentación no se muestra (entre ellas, por ejemplo, la relación de composición con la clase Documento).

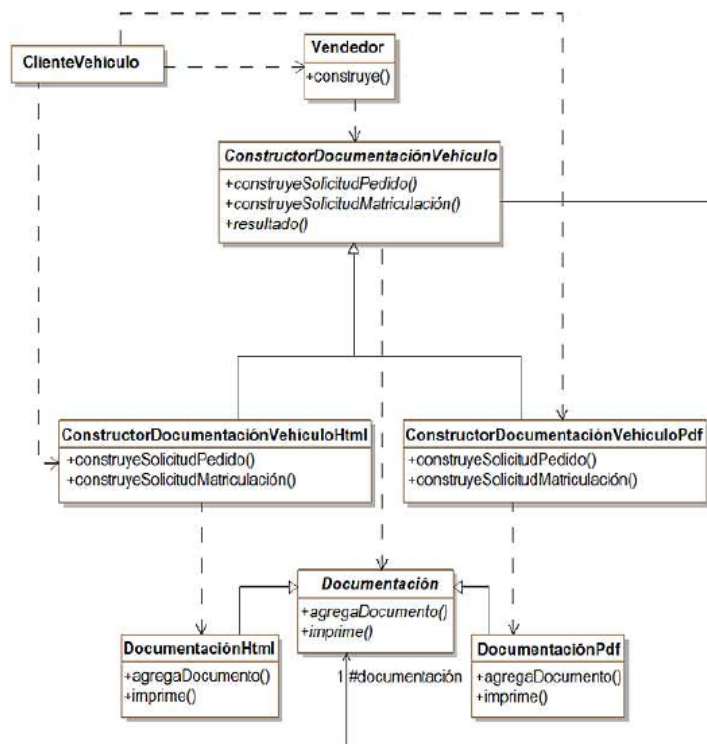


Figura 7: El patrón Builder aplicado a la generación de documentación

1.4.3. Estructura

La Figura 8 detalla la estructura genérica del patrón.

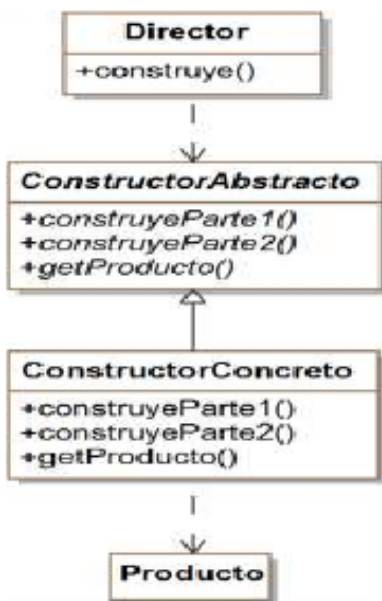


Figura 8: Estructura del patrón Builder

1.4.4. Participantes

- ConstructorAbstracto (ConstructorDocumentaciónVehiculo) es la clase que define la firma de los métodos que construyen las distintas partes del producto así como la firma del método que permite obtener el producto, una vez construido.
- ConstructorConcreto (ConstructorDocumentaciónVehiculoHtml y ConstructorDocumentaciónVehiculoPdf) es la clase concreta que implementa los métodos del constructor abstracto.
- Producto (Documentacion) es la clase que define el producto. Puede ser abstracta y poseer varias subclases concretas (DocumentacionHtml y DocumentacionPdf) en caso de implementaciones diferentes.
- Director es la clase encargada de construir el producto a partir de la interfaz del constructor abstracto.

1.4.5. Colaboradores

El cliente crea un constructor concreto y un director. El director construye, bajo demanda del cliente, invocando al constructor y reenvía el resultado al cliente.

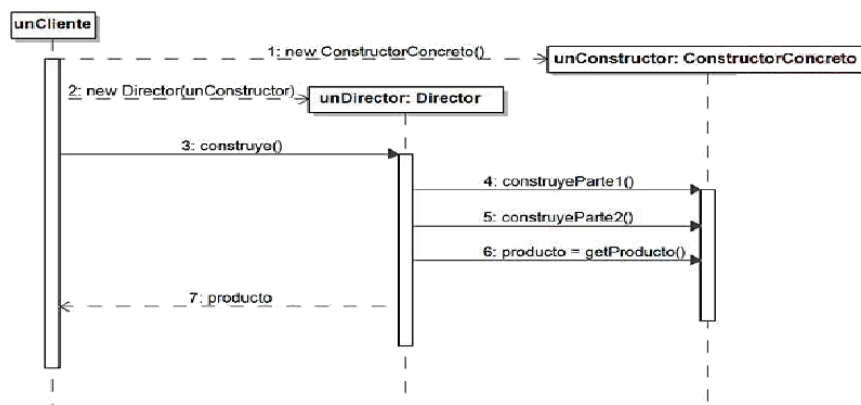


Figura 9: Diagrama de secuencia del patrón Builder

1.4.6. Dominio de Uso

El patrón se utiliza en los dominios siguientes:

- Un cliente necesita construir objetos complejos sin conocer su implementación.
- Un cliente necesita construir objetos complejos que tienen varias representaciones o implementaciones.

1.4.7. Ejemplo en Java

Se presenta a continuación un ejemplo de uso del patrón escrito en Java. El código Java correspondiente a la clase abstracta Documentacion y sus subclases aparece a continuación. Por motivos de simplicidad, los documentos son cadenas de caracteres para la documentación en formato HTML y PDF. El método imprime muestra las distintas cadenas de caracteres que representan los documentos.

```
1 import java.util.*;
2 public abstract class Documentacion {
3     protected List<String> contenido = new ArrayList<String>();
4
5     public abstract void agregaDocumento(String documento);
6     public abstract void imprime();
7 }
8
9 public class DocumentacionHtml extends Documentacion {
10     public void agregaDocumento(String documento) {
11         if (documento.startsWith("<HTML>"))
12             contenido.add(documento);
13     }
14     public void imprime() {
15         System.out.println("Documentacion HTML");
16         for (String s: contenido)
17             System.out.println(s);
18     }
19 }
20
21 public class DocumentacionPdf extends Documentacion {
22     public void agregaDocumento(String documento) {
23         if (documento.startsWith("<PDF>"))
24             contenido.add(documento);
25     }
26
27     public void imprime() {
28         System.out.println("Documentacion PDF");
29         for (String s: contenido)
30             System.out.println(s);
31     }
32 }
```

El código fuente de las clases que generan la documentación aparece a continuación.

```
1 public abstract class ConstructorDocumentacionVehiculo {
2     protected Documentacion documentacion;
3     public abstract void construyeSolicitudPedido(String nombreCliente);
4     public abstract void construyeSolicitudMatriculacion (String nombreSolicitante);
5     public Documentacion resultado() { return documentacion; }
6 }
7
8 public class ConstructorDocumentacionVehiculoHtml extends ConstructorDocumentacionVehiculo {
9     public ConstructorDocumentacionVehiculoHtml() {
10         documentacion = new DocumentacionHtml();
11     }
12     public void construyeSolicitudPedido(String nombreCliente) {
13         String documento;
14         documento = "<HTML>Solicitud de pedido Cliente: " + nombreCliente + "</HTML>";
15         documentacion.agregaDocumento(documento);
16     }
17     public void construyeSolicitudMatriculacion(String nombreSolicitante) {
18         String documento;
19         documento = "<HTML>Solicitud de matriculacion Solicitante: " + nombreSolicitante + "</HTML>";
20         documentacion.agregaDocumento(documento);
21     }
22 }
23
24 public class ConstructorDocumentacionVehiculoPdf extends ConstructorDocumentacionVehiculo {
25     public ConstructorDocumentacionVehiculoPdf() {
```

```

26     documentacion = new DocumentacionPdf();
27 }
28 public void construyeSolicitudPedido(String nombreCliente) {
29     String documento;
30     documento = "<PDF>Solicitud de pedido Cliente: " + nombreCliente + "</PDF>";
31     documentacion.agregaDocumento(documento);
32 }
33 public void construyeSolicitudMatriculacion(String nombreSolicitante) {
34     String documento;
35     documento = "<PDF>Solicitud de matriculacion Solicitante: " + nombreSolicitante + "</PDF>";
36     documentacion.agregaDocumento(documento);
37 }
38 }

```

La clase **Vendedor** se describe a continuación. Su constructor recibe como parámetro una instancia de: **ConstructorDocumentacionVehiculo**.

Observe que el método **construye** toma como parámetro la información del cliente, aquí limitada al nombre del cliente.

```

1 public class Vendedor {
2     protected ConstructorDocumentacionVehiculo constructor;
3
4     public Vendedor(ConstructorDocumentacionVehiculo constructor) {this.constructor = constructor;}
5
6     public Documentacion construye(String nombreCliente) {
7         constructor.construyeSolicitudPedido(nombreCliente);
8         constructor.construyeSolicitudMatriculacion(nombreCliente);
9         Documentacion documentacion = constructor.resultado();
10        return documentacion;
11    }
12 }

```

Por último, se proporciona el código Java del cliente del constructor, a saber la clase **ClienteVehiculo** que constituye el programa principal. El inicio de este programa solicita al usuario el constructor que debe utilizar, y se lo proporciona a continuación al vendedor.

```

1 import java.util.*;
2 public class ClienteVehiculo {
3     public static void main(String[] args) {
4         Scanner reader = new Scanner(System.in);
5         ConstructorDocumentacionVehiculo constructor;
6         System.out.print("Desea generar " + "documentacion HTML (1) o PDF (2):");
7         String seleccion = reader.next();
8         if (seleccion.equals("1")) {
9             constructor = new ConstructorDocumentacionVehiculoHtml();
10        }
11        else {
12            constructor = new ConstructorDocumentacionVehiculoPdf();
13        }
14        Vendedor vendedor = new Vendedor(constructor);
15        Documentacion documentacion = vendedor.construye("Martin");
16        documentacion.imprime();
17    }
18 }

```

Un ejemplo de ejecución para una documentación PDF sería:

```

Desea generar documentacion HTML (1) o PDF (2):2
Documentacion PDF
<PDF>Solicitud de pedido Cliente: Martin</PDF>
<PDF>Solicitud de matriculacion Solicitante: Martin</PDF>

```

Conforme a la solicitud del cliente, la documentación y sus documentos se han creado en formato PDF. Si el cliente solicitara su documentación en HTML, la salida sería la siguiente:

```

Desea generar documentacion HTML (1) o PDF (2):1
Documentacion HTML
<HTML>Solicitud de pedido Cliente: Martin</HTML>
<HTML>Solicitud de matriculacion Solicitante: Martin</HTML>

```

Existe otra forma de implementar el patrón Builder que lo puede encontrar en la página: https://www.danielsanmartin.cl/wiki/doku.php?id=wiki:pdd_3. Esta forma de implementación se considera estándar y tiene la ventaja de ser mas concisa. Como ejercicio, implemente la solución del Builder de este documento a la versión de la wiki.

1.5. Prototype

1.5.1. Descripción

El objetivo de este patrón es la creación de nuevos objetos mediante duplicación de objetos existentes llamados prototipos que disponen de la capacidad de clonación.

1.5.2. Ejemplo

Durante la compra de un vehículo, un cliente debe recibir una documentación compuesta por un número concreto de documentos tales como el certificado de cesión, la solicitud de matriculación o incluso la orden de pedido. Existen otros tipos de documentos que pueden incluirse o excluirse a esta documentación en función de las necesidades de gestión o de cambios de reglamentación. Introducimos una clase Documentación cuyas instancias son documentaciones compuestas por diversos documentos obligatorios. Para cada tipo de documento, incluimos su clase correspondiente.

A continuación creamos un modelo de documentación que consiste en una instancia particular de la clase Documentación y que contiene los distintos documentos necesarios, documentos en blanco. Llamamos a esta documentación “documentación en blanco”. De este modo definimos a nivel de las instancias, y no a nivel de las clases, el contenido preciso de la documentación que debe recibir un cliente. Incluir o excluir un documento en la documentación en blanco no supone ninguna modificación en su clase.

Una vez presentada la documentación en blanco, recurrimos al proceso de clonación para crear las nuevas documentaciones. Cada nueva documentación se crea duplicando todos los documentos de la documentación en blanco.

Esta técnica basada en objetos que poseen la capacidad de clonación utiliza el patrón Prototype, y los documentos constituyen los distintos prototipos.

La Figura 10 ilustra este uso. La clase Documento es una clase abstracta conocida por la clase Documentación. Sus subclases corresponden a los distintos tipos de documento. Incluyen el método duplica que permite clonar una instancia existente para obtener una nueva.

La clase Documentación también es abstracta. Posee dos subclases concretas:

- La clase DocumentaciónEnBlanco, que posee una única instancia que contiene todos los documentos necesarios (documentos en blanco). Esta instancia se manipula mediante los métodos incluye y excluye.
- La clase DocumentaciónCliente, cuyo conjunto de documentos se crea solicitando a la única instancia de la clase DocumentaciónEnBlanco la lista de documentos en blanco y agregándolos uno a uno tras haberlos clonado.

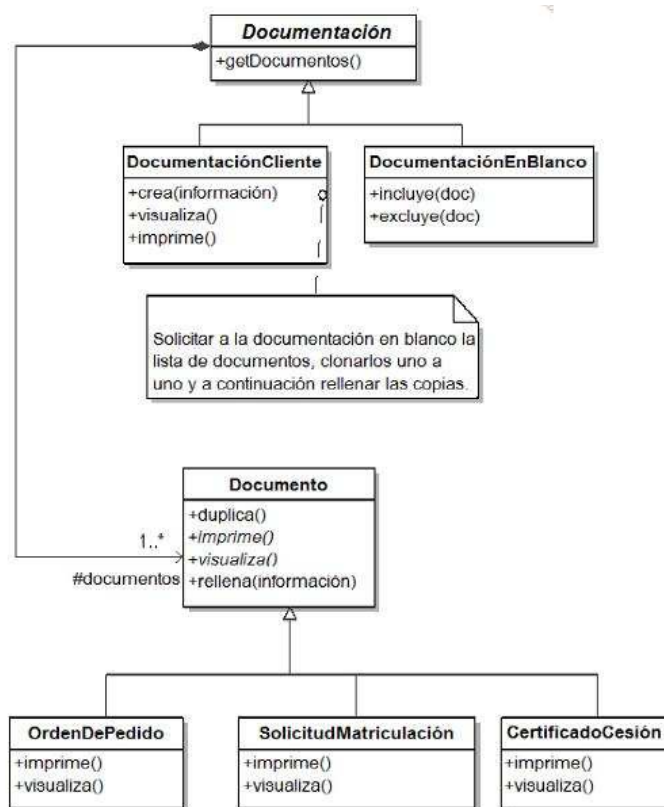


Figura 10: El patrón Prototype aplicado a la creación de documentación de contenido variable

1.5.3. Estructura

La figura 11 detalla la estructura genérica del patrón.

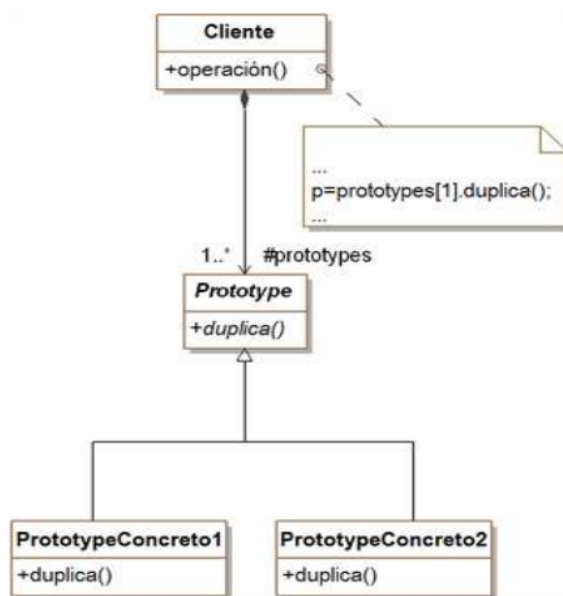


Figura 11: Estructura del patrón Prototype

1.5.4. Participantes

Los participantes del patrón son los siguientes:

- **Cliente** (`Documentacion`, `DocumentacionCliente`, `DocumentacionEnBlanco`) es una clase compuesta por un conjunto de objetos llamados prototipos, instancias de la clase abstracta **Prototype**. La clase **Cliente** necesita duplicar estos prototipos sin tener por qué conocer ni la estructura interna del **Prototype** ni su jerarquía de subclases.
- **Prototype** (`Documento`) es una clase abstracta de objetos capaces de duplicarse a sí mismos. Incluye la firma de método “duplica”.
- **PrototypeConcreto1** y **PrototypeConcreto2** (`OrdenPedido`, `SolicitudMatriculacion`, `CertificadoCesion`) son las subclases concretas de **Prototype** que definen completamente un prototipo e implementan el método `duplica`.

1.5.5. Colaboración

El cliente solicita a uno o varios prototipos que se dupliquen a sí mismos.

1.5.6. Dominio de Uso

El patrón Prototype se utiliza en los dominios siguientes:

- Un sistema de objetos debe crear instancias sin conocer la jerarquía de clases que las describe.
- Un sistema de objetos debe crear instancias de clases dinámicamente.
- El sistema de objetos debe permanecer simple y no incluir una jerarquía paralela de clases de fabricación.

1.5.7. Ejemplo de Java

El código fuente de la clase abstracta `Documento` y de sus subclases concretas aparece a continuación. Para simplificar, a diferencia del diagrama de clases, los métodos `duplica` y `rellena` se concretan en la clase `Documento`. El método `duplica` utiliza el método `clone` que proporciona Java.

El método `clone` de Java nos evita tener que copiar manualmente cada atributo. En consecuencia, es posible implementar el método `duplica` completamente en la clase abstracta `Documento`.

```

1 public abstract class Documento implements Cloneable {
2
3     protected String contenido = new String();
4     public Documento duplica() {
5
6         Documento resultado;
7
8         try {
9             resultado = (Documento)this.clone();
10        }
11        catch (CloneNotSupportedException exception) {
12            return null;
13        }
14        return resultado;
15    }
16
17    public void rellena(String informacion) {
18        contenido = informacion;
19    }
20    public abstract void imprime();
21    public abstract void visualiza();
22 }
23
24 public class OrdenDePedido extends Documento {
25     public void visualiza() {
26         System.out.println("Muestra la orden de pedido: " + contenido);
27     }
28     public void imprime() {
29         System.out.println("Imprime la orden de pedido: " + contenido);
30     }
31 }
32
33 public class SolicitudMatriculacion extends Documento {
34     public void visualiza() {
35         System.out.println("Muestra la solicitud de matriculacion: " + contenido);
36     }
37     public void imprime() {
38         System.out.println("Imprime la solicitud de matriculacion: " + contenido);
39     }
40 }
41
42 public class CertificadoCesion extends Documento {
43     public void visualiza() {
44         System.out.println("Muestra el certificado de cesion: " + contenido);
45     }
46     public void imprime() {
47         System.out.println("Imprime el certificado de cesion: " + contenido);
48     }
49 }

```

El código fuente de la clase abstracta Documentacion es el siguiente:

```

1 import java.util.*;
2 public abstract class Documentacion {
3     protected List<Documento> documentos;
4     public List<Documento> getDocumentos() {
5         return documentos;
6     }
7 }

```

El código fuente de la subclase DocumentacionEnBlanco de Documentacion aparece a continuación.

```

1 import java.util.*;
2 public class DocumentacionEnBlanco extends Documentacion {
3     private static DocumentacionEnBlanco _instance = null;
4     private DocumentacionEnBlanco() {
5         documentos = new ArrayList<Documento>();
6     }
7
8     public static DocumentacionEnBlanco Instance() {
9         if (_instance == null)
10             _instance = new DocumentacionEnBlanco();
11         return _instance;
12     }
13
14     public void incluye(Documento doc) {

```

```

15     documentos.add(doc);
16 }
17
18 public void excluye(Documento doc) {
19     documentos.remove(doc);
20 }
21 }

```

La subclase `DocumentacionCliente` escrita en Java aparece a continuación. Su constructor obtiene la lista de documentos de la documentación en blanco y a continuación los duplica, los rellena y los agrega a un contenedor de la documentación.

```

1 import java.util.*;
2 public class DocumentacionCliente extends Documentacion {
3     public DocumentacionCliente(String informacion) {
4         documentos = new ArrayList<Documento>();
5         DocumentacionEnBlanco documentacionEnBlanco = DocumentacionEnBlanco.Instance();
6         List<Documento> documentosEnBlanco = documentacionEnBlanco.getDocumentos();
7         for (Documento documento: documentosEnBlanco) {
8             Documento copiaDocumento = documento.duplica();
9             copiaDocumento.rellena(informacion);
10            documentos.add(copiaDocumento);
11        }
12    }
13    public void visualiza() {
14        for (Documento documento: documentos)
15            documento.visualiza();
16    }
17
18    public void imprime() {
19        for (Documento documento: documentos)
20            documento.imprime();
21    }
22 }

```

Por último, veamos el código fuente de la clase `Usuario` cuyo método principal (`main`) comienza construyendo la documentación en blanco y, en particular, su contenido. A continuación, este método crea y visualiza la documentación de ambos clientes.

```

1 public class Usuario {
2     public static void main(String[] args) {
3         //inicializacion de la documentacion en blanco
4         DocumentacionEnBlanco documentacionEnBlanco = DocumentacionEnBlanco.Instance();
5         documentacionEnBlanco.incluye(new OrdenDePedido());
6         documentacionEnBlanco.incluye(new CertificadoCesion());
7         documentacionEnBlanco.incluye(new SolicitudMatriculacion());
8         // creacion de documentacion nueva para dos clientes
9         DocumentacionCliente documentacionCliente1 = new DocumentacionCliente("Martin");
10        DocumentacionCliente documentacionCliente2 = new DocumentacionCliente("Simon");
11        documentacionCliente1.visualiza();
12        documentacionCliente2.visualiza();
13    }
14 }

```

El resultado de la ejecución es el siguiente:

```

Muestra la orden de pedido: Martin
Muestra el certificado de cesion: Martin
Muestra la solicitud de matriculacion: Martin
Muestra la orden de pedido: Simon
Muestra el certificado de cesion: Simon
Muestra la solicitud de matriculacion: Simon

```